

Sorting Optimizations for Lab 2

Insertion Sort

Armani: Half Exchanges (p 252 and exercise 2.1.25)

In the inner loop of insertion sort, do not use the exchange operation. Instead, before the inner loop, save value $a[i]$ to a temporary variable. Then, instead of swaps in the inner loop, move each item bigger than $a[i]$ up one place until you find where $a[i]$ goes, and put it in that spot.

Hanna: Binary Search Insertion

In the outer loop of insertion sort (looping over i), first do a binary search on range 0 to $i-1$ to find where to put $a[i]$ (searching for a position k where $a[k] \leq a[i] < a[k+1]$). Then shift up all values from k to $i-1$, and place the former $a[i]$ in the correct position.

Chumnuo: Insertion Sort with Sentinel (exercise 2.1.24)

Before the sort begins, find the minimum value and swap into index 0. Then remove the $j > 0$ check from the loop.

Selection Sort

Joel: Bidirectional Selection Sort

For each pass through the array, find both the max and min values, and swap into correct positions.

Shell Sort

Sam: Hibbards Sequence

The h -sequence for shell sort in the book is based on $3h+1$ (1, 4, 13, 40, 121, ...). Instead, try using sequence based on 2^{k-1} (1, 3, 7, 15, 31, ...). Also try $2^k + 1$ (1, 3, 5, 9, 17, 33, ...). It may be easier to put the sequence in a list, then iterate over it backwards.

Kayanna: Sedgewick Sequence

The h -sequence for shell sort in the book is based on $3h+1$ (1, 4, 13, 40, 121, ...). Instead, try using sequence based on: $4^k + 3 * 2^{k-1} + 1$ (8, 23, 77, 281, ...) Note that you must also finish with a pass of $h=1$. It may be easier to put the sequence in a list, then iterate over it backwards.

Ray: Half Exchanges

In the inner loop of shell sort, do not use the exchange operation. Instead, before the inner loop, save value $a[i]$ to a temporary variable. Then, instead of swaps, move each item bigger than $a[i]$ up one place until you find where $a[i]$ goes, and put it in that spot.

Merge Sort

Nate L.: Insertion Sort Small Cases:

If merge sort is called on a range that is less than 10, use insertion sort instead. (See code in **Insertion.java**) Try a few different values (besides 10) and try to find one that works the best.

Drew: Skip Merge Check

Note that after the two recursive merge sort calls, if the `max(left array) <= min(right array)`, then the two halves are already sorted and there is no need to merge. Add the check `a[mid] <= a[mid+1]` to skip the merge step if the two halves are in order.

Val: Swapping Auxilliary Array

If we swap back and forth which array we call **a** and which array we call **aux**, we can eliminate the need to copy values in **merge** function.

Change the two lines in merge sort to be:

```
sort(aux, a, lo, mid);  
sort(aux, a, mid + 1, hi);
```

Then delete the first loop in **merge** that copies values into aux

Brad: Half Copies

If you look careful at how the merge operation works, note that **k** (the position we are merging into) will never be greater than **j** (the current item from the right half that is being considered). Therefore, we don't have to worry about overwriting values in the right half, and we really only need the auxilliary array for the left array.

To fix this, the first loop inside merge should only copy over values **k** from **lo** to **mid**, and any time you have an array access involving **j**, take from **a[j]** instead of **aux[j]**.

Braxton: Quit Early if Sorted

If merge sort is called on a range of length 4 or less, manually check if the range is already sorted, and if so, immediately return. Try with a few different values (besides 4) and see which works the best.

Quick Sort

Luke: Median-Of-Three Partition (p 296)

If the range being partitioned has a length of at least 3, then find the median value between elements at indices **lo**, **mid**, **hi** and use that as the pivot (make sure to swap it into **a[lo]** before continuing the normal partition)

Zachary: Insertion Sort Small Cases (p 296)

If quick sort is called on a range that is less than 10, use insertion sort on that range instead. Try a few different values (besides 10) and try to find one that works the best.

Tadden: Defer Small Ranges (exercise 2.3.27)

If quick sort is called on a range that is less than 10, then stop sorting and return. Then, after the quick sort is finished, do insertion sort to clean up the nearly sorted array. Try a few different values (besides 10) and try to find one that works the best.

Caleb: Remove Sentinels (exercise 2.3.17)

You can remove the two sentinel checks `if (left >= hi)` and `if (right <= lo)` from partition if, before sorting, you find the max value in the array and put it at the very end (swap into position $n-1$).

Nate B.: Quit Early if Sorted

If quick sort is called on a range of length 4 or less, manually check if the range is already sorted, and if so, immediately return. Try with a few different values (besides 4) and see which works the best.